

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE



INFORME DE PRÁCTICA

Tema: Instalación de Entorno y Primer Script (PHP 8.2 y PERL/CGI)

Semestre: Quinto — Paralelo: “5A”

Estudiante: Jaime Josué Mariscal Cabrera

Asignatura: Aplicaciones Web

Docente: Ing. Guerrero Ulloa Gleiston Ciceron

Semana: Semana 5 (Gestión de Aula — Formativa)

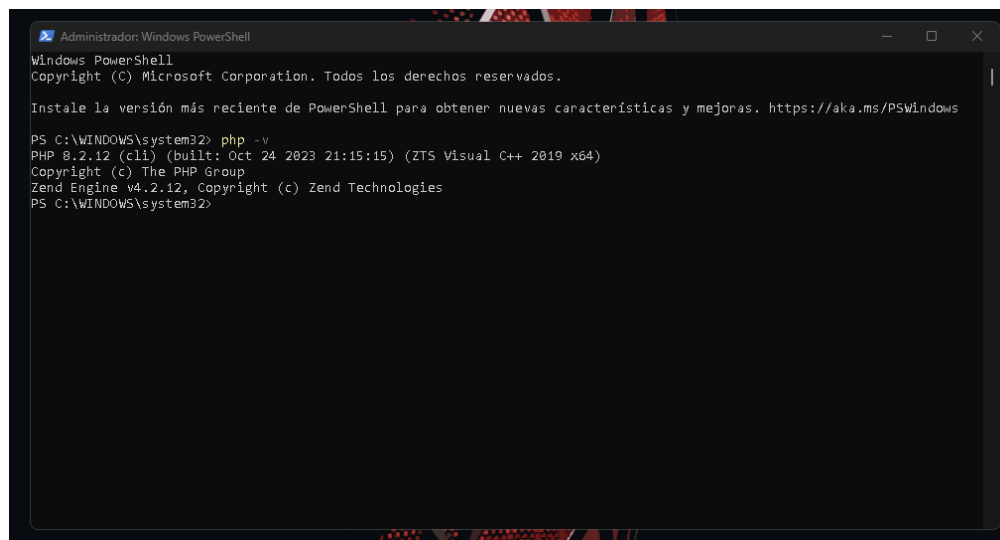
Fecha: 6 de junio de 2026

1 Instalación y Verificación del Entorno de Desarrollo

Se realizó la instalación del entorno local integrado utilizando XAMPP, seleccionando de manera obligatoria los componentes de Apache, PHP y el módulo de Perl para soporte CGI. Asimismo, se procedió a verificar la correcta habilitación de las directivas en el archivo de configuración `httpd.conf` de Apache, asegurando la carga de `mod_cgi.so` y el direccionamiento correcto del `ScriptAlias` hacia la carpeta `cgi-bin/`.

1.1 Verificación de Versiones en Terminal

A través de la interfaz de comandos del sistema, se ejecutaron las directivas de control para constatar el despliegue óptimo de los intérpretes de ejecución del lado del servidor.

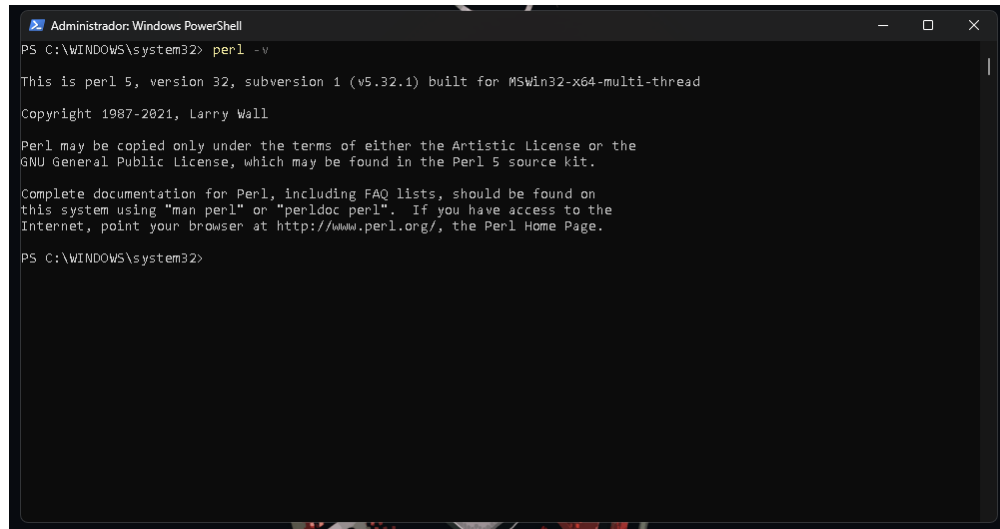


```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
PS C:\WINDOWS\system32>
```

Figura 1: Verificación de la versión de PHP 8.2.x en la terminal corporativa/local.



```
Administrador: Windows PowerShell
PS C:\WINDOWS\system32> perl -v

This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread
Copyright 1987-2021, Larry Wall

Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl".  If you have access to the
Internet, point your browser at http://www.perl.org/, the Perl Home Page.

PS C:\WINDOWS\system32>
```

Figura 2: Verificación del entorno de ejecución Perl (This is perl 5, version 32).

1.2 Estado del Panel de Control XAMPP

Se procedió con el levantamiento del servicio de Apache para habilitar la escucha activa en los puertos web predeterminados.

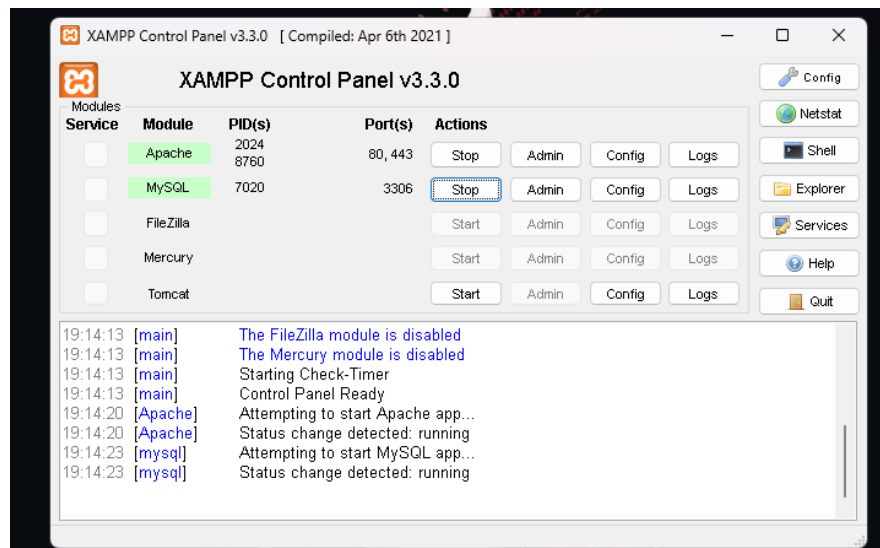


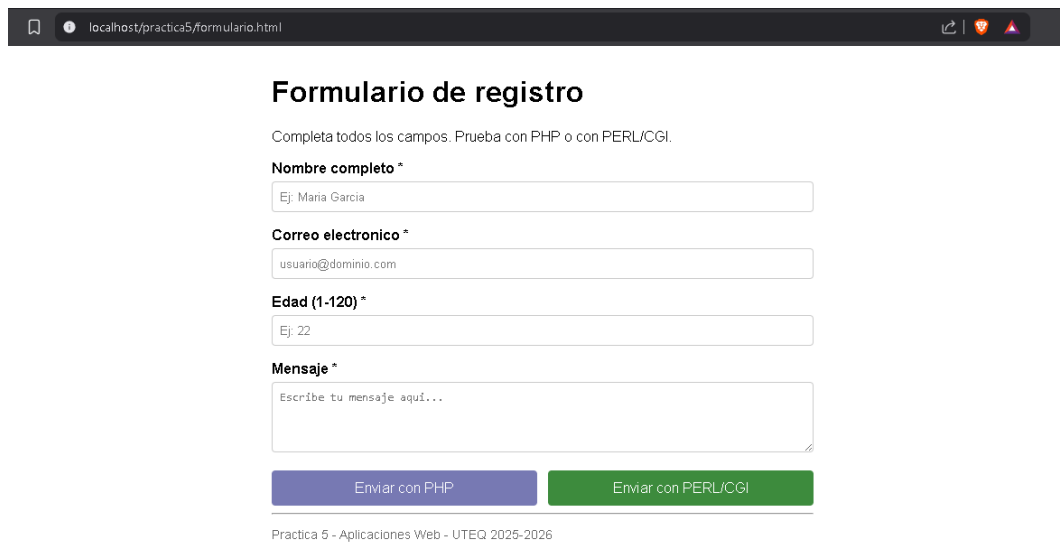
Figura 3: Panel de Control de XAMPP reflejando el servicio Apache en estado activo.

2 Estructura de Archivos y Formulario HTML Común

Para asegurar un análisis simétrico, se implementó una interfaz unificada mediante `formulario.html` en el directorio de trabajo local. Utilizando el atributo estándar `formaction` en los botones de envío, se redirige el flujo de datos POST hacia los entornos independientes de procesamiento distribuidos en la arquitectura local.

2.1 Visualización de la Interfaz Web

A continuación se detalla la presentación del formulario de captura desde el navegador web.



The screenshot shows a web browser window with the address bar displaying `localhost/practica5/formulario.html`. The page content is a registration form titled "Formulario de registro". Below the title, there is a instruction: "Completa todos los campos. Prueba con PHP o con PERL/CGI." The form contains four input fields: "Nombre completo *" with a placeholder "Ej: Maria Garcia", "Correo electronico *" with a placeholder "usuario@dominio.com", "Edad (1-120) *" with a placeholder "Ej: 22", and "Mensaje *" with a placeholder "Escribe tu mensaje aqui...". At the bottom of the form, there are two buttons: "Enviar con PHP" (purple) and "Enviar con PERL/CGI" (green). Below the buttons, there is a footer text: "Practica 5 - Aplicaciones Web - UTEQ 2025-2026".

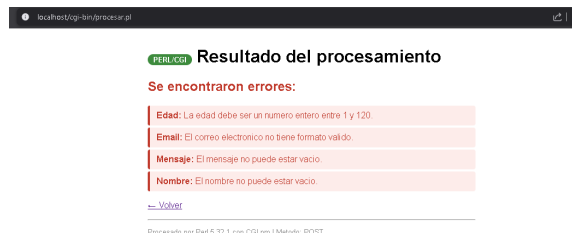
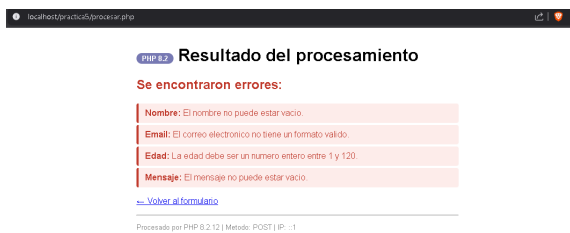
Figura 4: Interfaz del formulario común renderizado en el cliente web.

3 Evidencia de Verificación con Casos de Prueba

Se efectuó el despliegue de los 10 escenarios de control operacional de datos planteados en los requerimientos metodológicos, validando la resiliencia estructural de ambos backends.

3.1 Caso 1: Campos Vacíos

Envío del formulario sin rellenar datos para comprobar la captura global de errores de omisión.



3.2 Caso 2: Email sin Dominio

Evaluación de formato restrictivo usando la cadena `usuario@`.



3.3 Caso 3: Email sin Arroba

Validación de formato restrictivo usando la cadena `usuariodominio.com`.



3.4 Caso 4: Edad Negativa

Prueba perimetral de rango de enteros enviando un valor de `-20`.



3.5 Caso 5: Edad Demasiado Alta

Prueba perimetral de rango de enteros enviando un valor superior al límite permitido (200).



3.6 Caso 6: Edad No Numérica

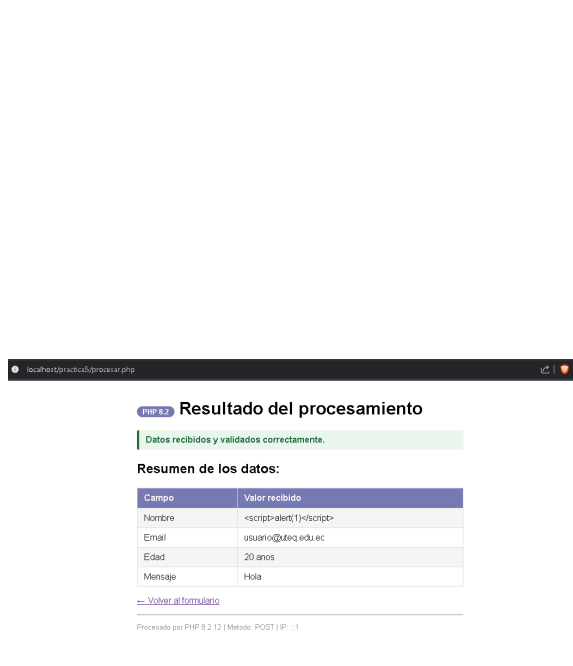
Inyección de cadenas de texto (e) en el parámetro correspondiente a enteros.



3.7 Caso 7: Intento de Cross-Site Scripting (XSS) en Nombre

Inyección de la etiqueta estructurada `<script>alert(1)</script>`. Se verifica el saneamiento mediante entidades HTML tanto en pantalla como en el código fuente del servidor.

Evidencias en PHP (Pantalla y Código Fuente)



PHP 8.2 Resultado del procesamiento

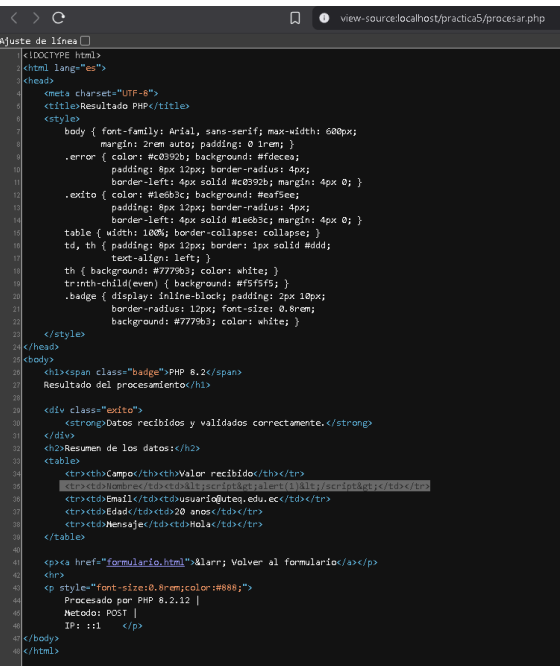
Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	<script>alert(1)</script>
Email	usuario@uteq.edu.ec
Edad	20 años
Mensaje	Hola

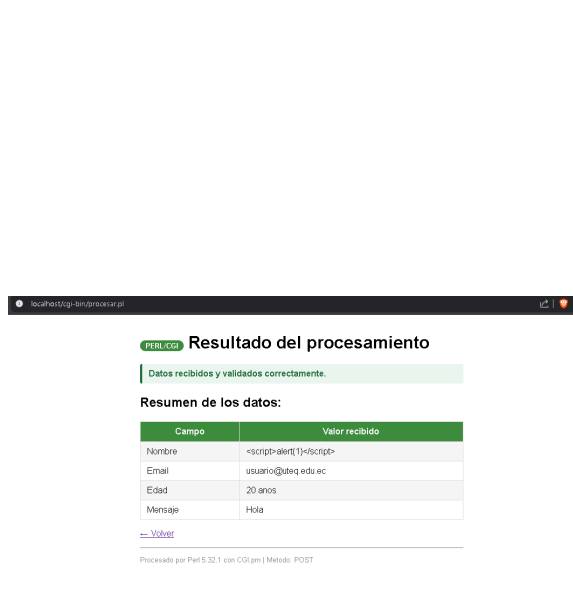
[Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ...



```
<?php
<doctype html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Resultado PHP</title>
<style>
body { font-family: Arial, sans-serif; max-width: 600px;
margin: 2rem auto; padding: 0 1rem; }
.error { color: #c0392b; background: #fde4e4;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #c0392b; margin: 4px 0; }
.exito { color: #1e8449; background: #eaf5ee;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #1e8449; margin: 4px 0; }
table { width: 100%; border-collapse: collapse; }
td, th { padding: 8px 12px; border: 1px solid #ddd; }
tr { text-align: left; }
tr:nth-child(even) { background: #f5f5f5; }
tr:nth-child(odd) { background: #fff; }
tr td { display: inline-block; padding: 2px 10px; }
tr td { border-radius: 12px; font-size: 0.8rem;
border-radius: 12px; font-size: 0.8rem;
background: #777; color: white; }
</style>
</head>
<body>
<div class="exito">
<strong>Datos recibidos y validados correctamente.</strong>
</div>
<div class="exito">
<strong>Resumen de los datos:</strong>
<table>
<tr>
<th>Campo</th>
<th>Valor recibido</th>
</tr>
<tr>
<td>Nombre</td>
<td><script>alert(1)</script>
</td>
</tr>
<tr>
<td>Email</td>
<td>usuario@uteq.edu.ec</td>
</tr>
<tr>
<td>Edad</td>
<td>20 años</td>
</tr>
<tr>
<td>Mensaje</td>
<td>Hola</td>
</tr>
</table>
</div>
<div>
<a href="formulario.html">Volver al formulario</a>
</div>
<p style="font-size: 0.8rem; color: #888;">
Procesado por PHP 8.2.12 |
Método: POST |
IP: ...
</p>
</body>
</html>
```

Evidencias en PERL (Pantalla y Código Fuente)



PERL CGI Resultado del procesamiento

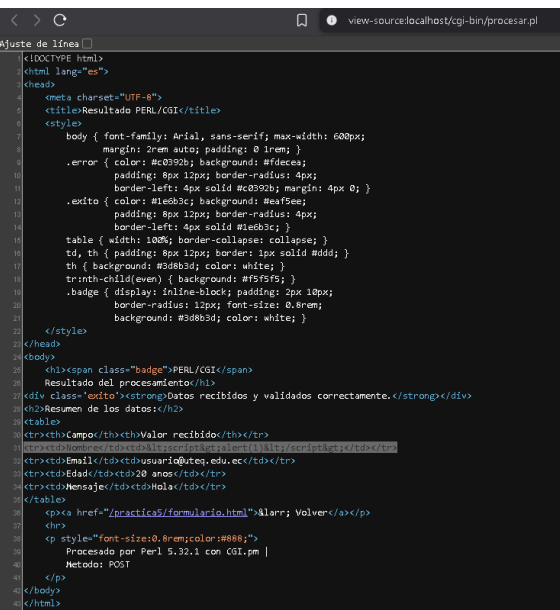
Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	<script>alert(1)</script>
Email	usuario@uteq.edu.ec
Edad	20 años
Mensaje	Hola

[Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST



```
<?perl
<doctype html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Resultado PERL/CGI</title>
<style>
body { font-family: Arial, sans-serif; max-width: 600px;
margin: 2rem auto; padding: 0 1rem; }
.error { color: #c0392b; background: #fde4e4;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #c0392b; margin: 4px 0; }
.exito { color: #1e8449; background: #eaf5ee;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #1e8449; margin: 4px 0; }
table { width: 100%; border-collapse: collapse; }
td, th { padding: 8px 12px; border: 1px solid #ddd; }
tr { text-align: left; }
tr:nth-child(even) { background: #f5f5f5; }
tr:nth-child(odd) { background: #fff; }
tr td { display: inline-block; padding: 2px 10px; }
tr td { border-radius: 12px; font-size: 0.8rem;
border-radius: 12px; font-size: 0.8rem;
background: #777; color: white; }
</style>
</head>
<body>
<div class="exito">
<strong>Datos recibidos y validados correctamente.</strong>
</div>
<div class="exito">
<strong>Resumen de los datos:</strong>
<table>
<tr>
<th>Campo</th>
<th>Valor recibido</th>
</tr>
<tr>
<td>Nombre</td>
<td><script>alert(1)</script>
</td>
</tr>
<tr>
<td>Email</td>
<td>usuario@uteq.edu.ec</td>
</tr>
<tr>
<td>Edad</td>
<td>20 años</td>
</tr>
<tr>
<td>Mensaje</td>
<td>Hola</td>
</tr>
</table>
</div>
<div>
<a href="/practica5/formulario.html">Volver</a>
</div>
<p style="font-size: 0.8rem; color: #888;">
Procesado por Perl 5.32.1 con CGI.pm |
Método: POST
</p>
</body>
</html>
```

3.8 Caso 8: Intento de XSS en Mensaje

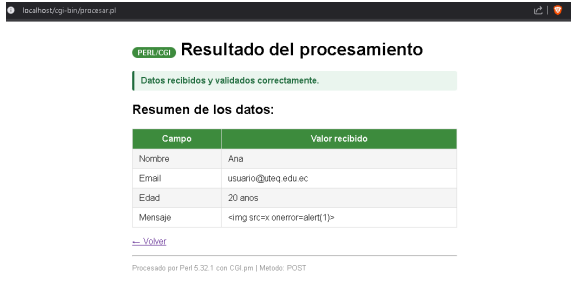
Inyección del elemento reactivo `<img src=x onerror=alert(1)>`. Se valida el correcto escape de caracteres especiales.

Evidencias en PHP (Pantalla y Código Fuente)



```
<?DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Resultado PHP</title>
<style>
body { font-family: Arial, sans-serif; max-width: 600px;
margin: 2rem auto; padding: 0 1rem; }
.error { color: #c0392b; background: #fdecea;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #c0392b; margin: 4px 0; }
.exito { color: #1e8449; background: #eaf5ee;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #1e8449; margin: 4px 0; }
table { width: 100%; border-collapse: collapse; }
td, th { padding: 8px 12px; border: 1px solid #ddd;
text-align: left; }
th { background: #7778b3; color: white; }
tr:nth-child(even) { background: #f5f5f5; }
.badge { display: inline-block; padding: 2px 10px;
border-radius: 12px; font-size: 0.8rem;
background: #7778b3; color: white; }
</style>
</head>
<body>
<div class="badge">PHP 8.2</span>
Resultado del procesamiento</div>
<div class="exito">
<strong>Datos recibidos y validados correctamente.</strong>
</div>
<h2>Resumen de los datos:</h2>
<table>
<tr><th>Campo</th><th>Valor recibido</th></tr>
<tr><td>Nombre</td><td>Ana</td></tr>
<tr><td>Email</td><td>usuario@uteq.edu.ec</td></tr>
<tr><td>Edad</td><td>20 años</td></tr>
<tr><td>Mensaje</td><td><img src=x onerror=alert(1)></td></tr>
</table>
<p><a href="formulario.html">&laquo; Volver al formulario</a></p>
<p style="font-size:0.8rem;color:#888;">
Procesado por PHP 8.2.12 |
Método: POST |
IP: ::1 </p>
</body>
</html>
```

Evidencias en PERL (Pantalla y Código Fuente)



```
<?DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Resultado PERL/CGI</title>
<style>
body { font-family: Arial, sans-serif; max-width: 600px;
margin: 2rem auto; padding: 0 1rem; }
.error { color: #c0392b; background: #fdecea;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #c0392b; margin: 4px 0; }
.exito { color: #1e8449; background: #eaf5ee;
padding: 8px 12px; border-radius: 4px;
border-left: 4px solid #1e8449; margin: 4px 0; }
table { width: 100%; border-collapse: collapse; }
td, th { padding: 8px 12px; border: 1px solid #ddd;
text-align: left; }
th { background: #3d8b3d; color: white; }
tr:nth-child(even) { background: #f5f5f5; }
.badge { display: inline-block; padding: 2px 10px;
border-radius: 12px; font-size: 0.8rem;
background: #3d8b3d; color: white; }
</style>
</head>
<body>
<div class="badge">PERL/CGI</span>
Resultado del procesamiento</div>
<div class="exito">
<strong>Datos recibidos y validados correctamente.</strong>
</div>
<h2>Resumen de los datos:</h2>
<table>
<tr><th>Campo</th><th>Valor recibido</th></tr>
<tr><td>Nombre</td><td>Ana</td></tr>
<tr><td>Email</td><td>usuario@uteq.edu.ec</td></tr>
<tr><td>Edad</td><td>20 años</td></tr>
<tr><td>Mensaje</td><td><img src=x onerror=alert(1)></td></tr>
</table>
<p><a href="practica5/formulario.html">&laquo; Volver</a></p>
<p style="font-size:0.8rem;color:#888;">
Procesado por Perl 5.32.1 con CGI.pm |
Método: POST
</p>
</body>
</html>
```


3.9 Caso 9: Datos Completamente Válidos

Procesamiento óptimo con parámetros que cumplen las especificaciones de negocio.

localhost/practica5/procesar.php

PHP 8.2 Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Ana
Email	usuario@uteq.edu.ec
Edad	20 años
Mensaje	Hola

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: :1

localhost/cgi-bin/procesar.pl

PERL CGI Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Ana
Email	usuario@uteq.edu.ec
Edad	20 años
Mensaje	Hola

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

3.10 Caso 10: Nombre con Caracteres Especiales Válidos

Envío de nombres compuestos con caracteres castellanos y guiones (María José Muñoz-López).

localhost/practica5/procesar.php

PHP 8.2 Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	María José Muñoz-López
Email	usuario@uteq.edu.ec
Edad	20 años
Mensaje	Hola

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: :1

localhost/cgi-bin/procesar.pl

PERL CGI Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	María José Muñoz-López
Email	usuario@uteq.edu.ec
Edad	20 años
Mensaje	Hola

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

4 Tabla Comparativa: PHP 8.2 vs. PERL 5.38 con CGI.pm

Se detalla el análisis de comportamiento técnico e infraestructura de ambos lenguajes de programación, incorporando métricas cualitativas derivadas del desarrollo experimental.

Tabla 1: Matriz comparativa de tecnologías del lado del servidor.

Criterio	PHP 8.2	PERL 5.38 con CGI.pm	Observación Personal
Integración con Apache	Módulo nativo (<code>mod_php</code>). El intérprete coexiste en el proceso de Apache. Alta velocidad.	CGI clásico. Lanza un proceso Perl nuevo por cada petición HTTP entrante.	PHP presenta un rendimiento y concurrencia superior por su acoplamiento directo.
Lectura de Formulario	Empleo de la función <code>filter_input(INPUT_POST, ...)</code> o el superglobal <code>\$_POST</code> .	Lectura orientada a objetos mediante la invocación de <code>\$cgi->param('campo')</code> .	PHP facilita la lectura lineal directa, mientras que Perl hereda un modelo de objetos clásico.
Saneamiento XSS	Invocación explícita de <code>htmlspecialchars(\$val, ENT_QUOTES, 'UTF-8')</code> .	Uso de la directiva integrada <code>\$cgi->escapeHTML(\$val)</code> .	Ambos entornos resuelven el problema convirtiendo caracteres en entidades seguras.
Validación de Email	Filtro nativo del motor web: <code>FILTER_VALIDATE_EMAIL</code> .	Evaluación por expresiones regulares manuales (Regex).	PHP ahorra líneas de código gracias a sus abstracciones lógicas nativas.
Cabecera HTTP	Gestión transparente/automática por Apache. Permite manipulación con <code>header()</code> .	Obligatoriedad de impresión explícita inicial: <code>\$cgi->header()</code> antes de salidas.	La omisión de la cabecera en Perl genera inmediatamente un crítico Error 500.
Curva de Aprendizaje	Media-Baja. Sintaxis intuitiva y documentación estandarizada.	Alta. Sintaxis compacta y diversa bajo la filosofía TM-TOWTDI.	PHP resulta óptimo para desarrollos rápidos de aplicaciones de software modernas.

5 Reflexión Final

Al realizar esta práctica, la diferencia que más me llamó la atención entre ambos lenguajes es su forma de ejecutarse en el servidor. PHP resulta ser mucho más rápido y eficiente porque trabaja directamente integrado como un módulo de Apache. En cambio, el modelo clásico de PERL con CGI es más demandante, ya que el servidor tiene que crear un proceso nuevo por cada petición que recibe. En un proyecto real con muchos usuarios conectados al mismo tiempo, esto definitivamente afectaría el rendimiento.

Por el lado de la seguridad, PHP nos facilita bastante el trabajo con funciones nativas como `filter_input()`, lo que ayuda a evitar errores humanos al momento de limpiar los datos de un formulario. Con Perl, el proceso es mucho más manual y te obliga a depender del uso estricto de expresiones regulares, lo que requiere más tiempo y cuidado para no dejar vulnerabilidades.